



P01-09 · T19–T20 Runbooks, DoD Evidence Pack & Phase Exit Report



The rule for this page: **a phase is not done when the code works, it is done when the evidence exists**. Every claim on this page has a command that produces a number, and every number gets pasted into the exit report. No checkbox is ticked from memory.

1. Runbook template — one file per error code

Every one of the 19 phase-01 error codes gets a file at `docs/runbooks/<CODE>.md`. The support site serves them at `aura.app/e/<CODE>`, which is the exact string embedded in `runbook_url`, so a photographer who sees a code can reach the page in one click.

```
<!-- docs/runbooks/AURA-IO-1003.md -->
---
code: AURA-IO-1003
title: A drive was disconnected during import
severity: run_blocking
recovery: retry
owner: SRC
phase: 01
last_reviewed: 2026-08-10
---

## What the photographer sees

> "The drive holding your photos was disconnected. AURA paused and kept
> everything it had already imported. Reconnect the drive and press
Resume."

## What actually happened
```

A read failed with a device-level error, or the volume serial number recorded on the source root no longer matches the mounted volume. AURA stopped the run at a transaction boundary rather than continuing and recording files as missing.

What AURA already did for you

- Committed every file that finished before the disconnection
- Left the run in `paused`, not `failed`
- Did **not** mark any file as absent – an unplugged drive is not a deleted file
- Released the task lease so the work is reclaimed on resume

Fix it in 30 seconds

1. Reconnect the drive. Use the same port if you can.
2. Wait for the operating system to show the volume.
3. In AURA, press **Resume**.

Import continues from where it stopped. Nothing is re-imported and nothing is duplicated, because identity is content-based.

If it keeps happening

Cause	How to tell	What to do
Bus-powered drive browning out	Happens on the same drive under load	Use a powered hub or a mains adapter
Cable or port fault	Happens at a random point every time	Swap the cable first, then the port
Drive going bad	The OS logs read errors too	Copy the card to internal storage and import from there
Aggressive sleep settings	Happens after a long idle pause	Disable drive sleep while importing

```
## Copy the card first if the drive is failing
```

When a card or drive is genuinely unhealthy, copy it to internal storage with a tool that verifies, then import from the copy. AURA's `copy\_verified` import mode does this for you: it copies, re-hashes the copy, and refuses to record the file unless the hashes match.

```
## For support and engineers
```

- Log query: `grep AURA-IO-1003 ~/.../logs/aura.log`
- Catalog query:

```
SELECT root_id, volume_serial, is_removable, last_seen_at  
FROM source_root WHERE project_id = ?;
```

- The paused run is recoverable indefinitely; there is no expiry
- Related codes: `AURA-IO-1006` (unreadable file), `AURA-JOB-7001` (lease expired)
- Test that proves [this behaviour](#): `tests/chaos/volume.rs::volume\_disconnect pauses and resumes without data loss`

The 19 runbooks and their one-line promise

Code	Runbook title	The promise the page must keep	Owner
AURA-IO-1001	A file AURA expected is no longer there	Tell them which folder, and that the catalog is unharmed	SRC
AURA-IO-1002	AURA cannot write where you asked	Name a folder that will work, per platform	SRC
AURA-IO-1003	A drive was disconnected during import	Reconnect and resume; nothing lost, nothing duplicated	SRC
AURA-IO-1004	The disk is full	How much is needed, and what is safe to delete	SRC
AURA-IO-1005	A file path is too long for this system	Exactly how to shorten it; Windows-specific guidance	SRC

Code	Runbook title	The promise the page must keep	Owner
AURA-IO-1006	A file could not be read	The photo is quarantined, not lost; how to retry one file	SRC
AURA-IO-1007	A file changed while AURA was reading it	Usually a sync client; how to check and retry	SRC
AURA-IO-1008	The catalog cannot live in a synced folder	Why syncing corrupts a database; where to put it instead	SRC
AURA-IO-1010	A file is empty	Almost always a bad card or an interrupted copy; recovery steps	SRC
AURA-DB-3001	The catalog could not be opened	The three causes, in the order to check them	SRC
AURA-DB-3002	An update to the catalog did not finish	The backup is already made; how to restore it in two steps	SRC
AURA-DB-3003	The catalog file is damaged	Restore from <code>backups/</code> ; what is recoverable and what is not	SRC
AURA-DB-3004	This catalog was made by a newer AURA	Update the app; never downgrade a catalog	SRC
AURA-DB-3005	AURA is already running	Find the other window; what to do after a hard crash	SRC
AURA-DB-3006	The safety backup could not be made	Why AURA refuses to migrate without one	SRC
AURA-JOB-7001	A task was interrupted and will be retried	Informational; no action needed; where to see it	TLC
AURA-JOB-7002	A photo failed three times and was set aside	Where quarantine lives; how to retry after fixing the cause	TLC
AURA-JOB-7003	Internal: a task graph problem	A bug, not a user error; what to attach to a report	TLC
AURA-JOB-7004	A task's result was discarded	Informational; explains a repeated step in the log	TLC
AURA-JOB-7005	AURA stopped to protect your catalog	What to send us; the catalog is intact and openable read-only	TLC

cargo xtask runbooks --check

```
// xtask/src/runbooks.rs
```

```
/// A runbook gate is worth more than a runbook policy. This runs in the PR lane,
```

```

/// so a new error code without a runbook cannot be merged.
pub fn check() -> anyhow::Result<> {
    let registry = parse_errors_toml("crates/aura-core/errors.toml")?;
    let mut problems = Vec::new();

    for entry in &registry {
        let path = PathBuf::from(format!("docs/runbooks/{}.md", entry.code));
        if !path.exists() {
            problems.push(format!("{}", entry.code, path.display()));
            continue;
        }
        let body = std::fs::read_to_string(&path)?;
        let front = parse_front_matter(&body)?;

        // The front matter must agree with the registry, or the support site
        // shows a different severity from the app.
        if front.severity != entry.severity {
            problems.push(format!("{}", entry.code, front.severity, entry.severity));
        }
        if front.recovery != entry.recovery {
            problems.push(format!("{}", entry.code));
        }
        for required in ["## What the photographer sees",
                        "## What actually happened",
                        "## Fix it in 30 seconds",
                        "## For support and engineers"] {
            if !body.contains(required) {
                problems.push(format!("{}", entry.code));
            }
        }
        // A runbook nobody has read in a year is worse than none, b

```

```

because it is
    // trusted and wrong.
    if front.last_reviewed_older_than_days(365) {
        problems.push(format!("{}: last_reviewed is over a year
old", entry.code));
    }
}

// The reverse direction: a runbook for a code that no longer ex
ists is dead
// documentation that will eventually mislead someone.
for file in std::fs::read_dir("docs/runbooks")? {
    let name = file?.file_name().to_string_lossy().replace(".m
d", "");
    if name.starts_with("AURA-") && !registry.iter().any(|e| e.c
ode == name) {
        problems.push(format!("{name}: runbook exists but the co
de is not registered"));
    }
}

if !problems.is_empty() {
    for p in &problems { eprintln!(" {p}"); }
    anyhow::bail!("{:} runbook problem(s)", problems.len());
}
println!("{:} runbooks checked, all consistent with the registr
y", registry.len());
Ok(())
}

```

2. Operational runbooks — the four that are not error codes

File	Question it answers	Must contain
<code>docs/runbooks/OPS-restore-backup.md</code>	"My catalog is damaged, how do I get yesterday back?"	The exact <code>backups/</code> naming scheme, the two-command restore, and how to verify the restored file with <code>integrity_check</code> before trusting it
<code>docs/runbooks/OPS-move-catalog.md</code>	"I want the catalog on my new SSD."	Close AURA first; move three files (<code>catalog.sqlite</code> , <code>-wal</code> , <code>-shm</code>); never copy

File	Question it answers	Must contain
		a live WAL; how the source roots stay valid
<code>docs/runbooks/OPS-collect-diagnostics.md</code>	"Support asked me to send logs."	One command that produces a redacted bundle; exactly what is inside it; the promise that no client path or image is included
<code>docs/runbooks/OPS-retry-quarantine.md</code>	"Twelve photos were set aside. Now what?"	How to see the reason per file, fix the cause, and retry only those files without re-importing the wedding

```
# The diagnostics bundle, since it is the one support will ask for daily.
# Redaction is not optional and not a flag: there is no unredacted mode.
cargo run -p aura-cli -- diagnostics --out ~/Desktop/aura-diagnostics.zip

# Contents, and nothing else:
#   app-version.txt      version, commit, build date, OS, CPU, RAM
#   schema.txt           schema_version, table row counts (counts, not rows)
#   integrity.txt        output of PRAGMA integrity_check and foreign_key_check
#   errors.csv           code, count, first_seen, last_seen – no paths
#   budgets.csv          the last local budget measurements, if any
#   log-tail.txt         last 5,000 log lines, already redacted at write time
#   consent.txt          which consent flags are on, per project id (no names)
```

3. The DoD evidence pack — `docs/phase-exit/PHASE-01-EVIDENCE.md`

One row per requirement. `Command` is what produces the proof; `Evidence` is where the pasted output lives. A row with an empty Evidence cell blocks the phase.

#	Requirement	Command that proves it	Evidence artefact
1	Workspace builds clean on 3 platforms	<code>cargo build --workspace --all-targets</code>	<code>evidence/01-build-{linux,mac,win}.txt</code>
2	Zero clippy warnings, zero banned patterns	<code>cargo clippy ... -- -D warnings</code> and <code>./scripts/check-banned.sh</code>	<code>evidence/02-lint.txt</code>
3	All 7 frozen contracts digested and unchanged	<code>cargo xtask contracts --check</code>	<code>evidence/03-contracts.txt</code> • <code>contracts.lock</code>
4	Rust and TypeScript IPC types identical	<code>cargo xtask ipc-types --check</code>	<code>evidence/04-ipc-parity.txt</code>
5	Catalog schema v1 applied; integrity clean	<code>cargo test -p aura-catalog</code>	<code>evidence/05-catalog.txt</code>
6	Second instance refused	<code>cargo test -p aura-catalog second_writer</code>	<code>evidence/06-single-writer.txt</code>
7	4,000-file import completes inside budget	<code>cargo test -p aura-perf budget_import_4000 -- --ignored</code>	<code>evidence/07-import-4000.txt</code>
8	Re-import imports zero files, catalog digest identical	<code>cargo test -p aura-ingest idempotence</code>	<code>evidence/08-idempotence.txt</code>
9	Duplicate folder produces one photo per photograph	<code>cargo test -p aura-ingest duplicate_folder</code>	<code>evidence/09-dedup.txt</code>
10	All 60 hostile files behave as declared in <code>manifest.toml</code>	<code>cargo test --test hostile</code>	<code>evidence/10-hostile.txt</code>
11	All 8 kill points converge to the	<code>cargo test --test chaos_resume -- --ignored</code>	<code>evidence/11-chaos-kill.txt</code>

#	Requirement	Command that proves it	Evidence artefact
	same catalog digest		
12	Volume yank pauses and resumes with no loss	<code>cargo test --test chaos_volume -- --ignored</code>	<code>evidence/12-volume.txt</code>
13	Disk full halts cleanly; catalog reopens	<code>cargo test --test chaos_diskfull -- --ignored</code>	<code>evidence/13-diskfull.txt</code>
14	Resume after kill under 5 s	<code>cargo test -p aura-perf budget_resume -- --ignored</code>	<code>evidence/14-resume.txt</code>
15	Peak RSS under 1.2 GB	<code>cargo test -p aura-perf budget_peak_rss -- --ignored</code>	<code>evidence/15-rss.txt</code>
16	Cold launch under 2.5 s on all 3 platforms	<code>cargo test -p aura-perf budget_cold_launch -- --ignored</code>	<code>evidence/16-launch.txt</code>
17	Cancel acknowledged under 500 ms	<code>cargo test -p aura-perf budget_cancel -- --ignored</code>	<code>evidence/17-cancel.txt</code>
18	4,000-cell grid: no frame over 16.7 ms	<code>npm run test:perf in ui</code>	<code>evidence/18-grid-trace.json</code>
19	No client path in any log or telemetry artefact	<code>cargo test --test privacy</code>	<code>evidence/19-privacy.txt</code>
20	Zero network calls in the whole phase	<code>cargo test no_network_syscall</code>	<code>evidence/20-network.txt</code>
21	New project consent all-false after a DB round trip	<code>cargo test consent_defaults_to_denied</code>	<code>evidence/21-consent.txt</code>
22	Every one of the 19 codes has a consistent runbook	<code>cargo xtask runbooks --check</code>	<code>evidence/22-runbooks.txt</code>

#	Requirement	Command that proves it	Evidence artefact
23	Every code is asserted by at least one test	<code>cargo test every_error_code_is_asserted_somewhere</code>	<code>evidence/23-code-coverage.txt</code>
24	Coverage at or above 85 % lines	<code>cargo llvm-cov --fail-under-lines 85</code>	<code>evidence/24-coverage.txt</code>
25	No licence or advisory violations	<code>cargo deny check</code>	<code>evidence/25-deny.txt</code>
26	Zero flakes in 20 consecutive runs	the flake-hunt lane, twice	<code>evidence/26-flake.txt</code>
27	Fixture digest identical on all 3 platforms	<code>cargo xtask fixtures --verify-digest</code>	<code>evidence/27-fixture-digest.txt</code>
28	The manual 15-minute acceptance walkthrough	a human, following section 4	<code>evidence/28-walkthrough.md</code> • screen recording

```
#!/usr/bin/env bash
# scripts/collect-evidence.sh
# Produces every artefact in the table above, in order, into docs/phase-exit/evidence/.
#
# Why a script rather than a checklist: a human running 28 commands by hand will
# skip the slow ones on the day the phase is late, which is exactly the day the
# evidence matters most.
set -uo pipefail
OUT="docs/phase-exit/evidence"
mkdir -p "$OUT"

record() {
    local file="$1"; shift
    echo "==> $file"
    {
```

```

echo "# command: $*"
echo "# host: $(uname -a)"
echo "# commit: $(git rev-parse HEAD)"
echo "# utc: $(date -u +%Y-%m-%dT%H:%M:%SZ)"
echo
"$@" 2>&1
echo
echo "# exit: $?"
} | tee "$OUT/$file"
}

record 01-build.txt cargo build --workspace --all-targets
record 02-lint.txt bash -c 'cargo clippy --workspace --all-targets -- -D warnings && ./scripts/check-banned.sh'
record 03-contracts.txt cargo xtask contracts --check
record 04-ipc-parity.txt cargo xtask ipc-types --check
record 05-catalog.txt cargo test -p aura-catalog
record 08-idempotence.txt cargo test -p aura-ingest --test idempotence
record 10-hostile.txt cargo test --test hostile
record 11-chaos-kill.txt cargo test --test chaos_resume -- --ignored --nocapture
record 19-privacy.txt cargo test --test privacy -- --nocapture
record 22-runbooks.txt cargo xtask runbooks --check
record 24-coverage.txt cargo llvm-cov --workspace --summary-only --fail-under-lines 85
record 25-deny.txt cargo deny check
record 27-fixture-digest.txt cargo xtask fixtures --verify-digest

# Perf rows are separate because they need the Full corpus and the pinned machine.
if [ "${AURA_PERF_MACHINE:-}" = "t-typ" ]; then
    record 07-import-4000.txt cargo test --release -p aura-perf budget_import_4000 -- --ignored --nocapture
    record 14-resume.txt cargo test --release -p aura-perf budget_resume -- --ignored --nocapture
    record 15-rss.txt cargo test --release -p aura-perf budget_peak_rss -- --ignored --nocapture
else

```

```

    echo "SKIPPED perf rows: set AURA_PERF_MACHINE=t-typ on the refere
nce machine" \
    | tee "$OUT/PERF-SKIPPED.txt"
fi

echo
echo "evidence written to $OUT"
echo "rows still missing evidence:"
cargo xtask evidence --check # cross-references the table with the
files on disk

```

4. The manual walkthrough — 15 minutes, done by a human

Automated tests cannot tell you the app feels trustworthy. This is done once per phase, on each platform, by someone who did not write the code.

Step	Action	Pass condition
1	Launch on a machine that has never run AURA	Welcome screen inside 2.5 s; no account prompt; no network indicator activity
2	Try to put the catalog in a Dropbox folder	Refused with AURA-I0-1008 and the exact wording from P01-06
3	Put the catalog in Pictures; create a project named Priyanka & Rohan	Created; every consent toggle visibly off
4	Import a folder of 400 real RAWs from a card reader	Live count rises; no fake percentage during scan; ETA appears only once it is steady
5	Mid-import, unplug the card reader	Paused, not failed, with the reconnect message; grid keeps what it had
6	Reconnect and press Resume	Continues; final count equals the folder count; no duplicates
7	Import the same folder again	"0 new files" and a visible "already in your catalog" count
8	Copy the folder to a second location and import that	File count rises, photo count does not
9	Start an import and press Cancel after 3 seconds	Stops within half a second; partial work retained; state says cancelled
10	Kill the app from the task manager mid-import, relaunch	Resumes within 5 s without asking anything; counts converge

Step	Action	Pass condition
11	Launch a second copy of AURA	Refused immediately with AURA-DB-3005 ; the first is undisturbed
12	Scroll the grid hard from top to bottom	Smooth; no white flashes; no stall over a fifth of a second
13	Open the quarantine list	Each entry has a plain-language reason and a code
14	Run the diagnostics bundle and read every file in it	No client path, no client name, no image anywhere in the zip
15	Open the log file and search for the project name	Zero matches

5. **docs/phase-exit/PHASE-01.md** — the exit report template

```
# PHASE-01 Exit Report – Project Foundation, Catalog & Wedding Project Ingest
```

```
| Field | Value |
| --- | --- |
| Phase | 01 of 30 |
| Epic | E1 Foundation |
| Planned | 2 weeks |
| Actual | __ days |
| Commit | `__` |
| Contracts frozen | 7 |
| Error codes registered | 19 |
| Tests added | __ |
| Coverage | __ % lines / __ % branches |
| Signed off | CTO __ / TLC __ / QAL __ / PERF __ |
```

```
## 1. What now works that did not before
```

A photographer can point AURA at a card, watch an honest progress bar, kill the process at any moment, relaunch, and reach a complete deduplicated catalog with zero duplicate rows. State this in one paragraph, in the photographer's words,

not the architecture's.

2. Budget results

Budget	Target	Ceiling	Measured	Verdict
Import 4,000 files	300 s	540 s	__	
Hash throughput	700 MB/s	400 floor	__	
Catalog open 200k	250 ms	600 ms	__	
Peak RSS	700 MB	1200 MB	__	
Resume after kill	2 s	5 s	__	
Cold launch	1.2 s	2.5 s	__	
Cancel ack	200 ms	500 ms	__	
Migration 200k	1 s	5 s	__	
Worst grid frame	8 ms	16.7 ms	__	
list_grid 4,000	60 ms	150 ms	__	

These numbers become `perf/baseline.json`. Every later phase is measured against

them, so an error here is inherited by all 29 remaining phases.

3. Evidence pack

All 28 rows of `PHASE-01-EVIDENCE.md` filled, with artefacts in `evidence/`.

Any row marked SKIPPED must name who accepted the risk and why.

4. Decisions taken, as ADRs

ADR	Decision	Alternative rejected
ADR-0001	Content hash is identity	Path-based identity
ADR-0002	`synchronous = FULL`, not NORMAL	15 % faster imports
ADR-0003	Single-writer actor	Multi-writer with `busy_timeout`
ADR-0004	Advisory file lock, not a PID file	PID files, which lie after a crash
ADR-0005	`STRICT` tables everywhere	SQLite's default flexible

```

typing |
| ADR-0006 | Sorted scan order is a contract | Filesystem order |
| ADR-0007 | ETA withheld unless defensible | Always showing an esti
mate |
| ADR-0008 | Quarantine, not failure, for bad files | Failing the wh
ole run |

```

5. Known limitations, deliberately shipped

List them plainly. Phase 01 does not decode RAWs, does not build pre views beyond embedded Tier 1, has no face or scene understanding, and has no clou d path at all. Anything a demo cannot do must appear here so nobody promises i t.

6. What phase 02 inherits

```

| Artefact | Guarantee to phase 02 |
| --- | --- |
| `photo` / `photo_file` schema | Frozen; phase 02 adds tables, neve
r alters these |
| `AuraError` + 19 codes | Frozen; phase 02 adds codes in its own ra
nges |
| Job graph and leases | Frozen; phase 02 adds stages, not mechanics
|
| `ImportedFile` contract | Frozen; carries `embedded_preview_offset
` for Tier 1 |
| Budget harness | Reused as-is; phase 02 adds rows to `budgets.toml
` |
| Fixture generator | Extended with real decodable RAWs for decode t
ests |

```

7. Risks carried forward

```

| Risk | Likelihood | Impact | Mitigation now | Owner |
| --- | --- | --- | --- | --- |
| Real RAW EXIF variance exceeds fixtures | Medium | Medium | Phase
02 adds a real-camera corpus | DATA |

```

```
| Windows long-path edge cases in the wild | Medium | High | `\\?\`  
prefix plus quarantine, both tested | SRC |  
| SQLite contention once GPU stages land | Low | High | Single-write  
r actor already in place | SRC |  
| Fixture RAWs are not decodable | High | Low | Known and accepted;  
phase 02 supplies real files | QAL |
```

8. Sign-off

No phase-02 work starts until every box below is ticked by the named role.

- [] CT0 – architecture and the 7 frozen contracts
- [] TLC – error taxonomy, job graph, resume semantics
- [] SRC – catalog, migrations, single writer
- [] SFE – shell, IPC, honest progress, grid
- [] QAL – test suite, chaos, evidence pack
- [] PERF – budgets measured on the reference machine
- [] DEVOPS – CI lanes green, nightly green, branch protection on
- [] SEC – no path in logs, no network call, consent defaults denied

6. **cargo xtask evidence --check** — the gate on the gate

```
// xtask/src/evidence.rs  
  
/// Parses the evidence table out of PHASE-01-EVIDENCE.md and verifies that every  
/// row has a real, non-empty, recent artefact on disk.  
///  
/// This exists because "paste the output" is the step humans skip.  
A phase that  
/// cannot prove itself is not finished, and the check must be mechanical.  
pub fn check(phase: &str) -> anyhow::Result<()> {  
    let doc = std::fs::read_to_string(format!("docs/phase-exit/PHASE-  
-{{phase}}-EVIDENCE.md"))?;  
    let rows = parse_markdown_table(&doc)?;
```



```

let head = current_commit()?;
let mut missing = Vec::new();

for row in &rows {
    let path = PathBuf::from(format!("docs/phase-exit/{}", row.artefact));
    match std::fs::metadata(&path) {
        Err(_) => missing.push(format!("row {}: {} does not exist", row.number, row.artefact)),
        Ok(meta) if meta.len() < 200 =>
            missing.push(format!("row {}: {} is suspiciously small ({} bytes)",
                                row.number, row.artefact, meta.len())),
        Ok(_) => {
            let body = std::fs::read_to_string(&path)?;
            // Evidence from a different commit is evidence for
            different code.
            if !body.contains(&head) {
                missing.push(format!("row {}: {} was produced at
a different commit",
                                row.number, row.artefact));
            }
            // A pasted failure is not evidence of success.
            if body.contains("# exit: 0") == false && body.contains("# exit:") {
                missing.push(format!("row {}: {} records a non-zero exit",
                                row.number, row.artefact));
            }
        }
    }
}

if !missing.is_empty() {
    for m in &missing { eprintln!("{}", m); }
    anyhow::bail!("{} of {} evidence rows incomplete", missing.len(), rows.len());
}
println!("all {} evidence rows present at commit {head}", rows.len());

```

```
en());  
    Ok(())  
}
```

7. Acceptance criteria for T19–T20

- ☐ 19 error-code runbooks exist, one per registered code, each with all four required sections
- ☐ Front matter `severity` and `recovery` match `errors.toml` for every code; `cargo xtask runbooks --check` passes
- ☐ A runbook for an unregistered code fails the check, demonstrated
- ☐ Removing one runbook fails the PR lane, demonstrated
- ☐ The four `OPS-*` runbooks exist and were followed end to end by someone who did not write them
- ☐ `aura-cli diagnostics` produces the seven-file bundle; every file inspected; zero client paths, names or images
- ☐ `PHASE-01-EVIDENCE.md` exists with all 28 rows; `scripts/collect-evidence.sh` produces every non-perf artefact in one run
- ☐ `cargo xtask evidence --check` passes; deleting one artefact makes it fail, demonstrated
- ☐ Every artefact records host, commit and UTC timestamp, and every one is from the same commit
- ☐ The perf rows are filled from the pinned reference machine, not a laptop; the machine is named in the report
- ☐ `perf/baseline.json` committed from those numbers
- ☐ The 15-step manual walkthrough completed on Windows, macOS and Linux by someone who did not write the code; a screen recording attached per platform
- ☐ All 8 ADRs written and linked from the exit report
- ☐ Known limitations section names every capability a demo must not promise
- ☐ All 8 sign-off boxes ticked by the named roles
- ☐ Phase 02 is not started until this page's checklist is complete

8. Claude Code prompt for T19–T20

Act as `DOC` for the runbooks, `QAL` for the evidence pack, and `PM` for the exit report. Implement T19 and T20 of PHASE-01 from this page.

Write all 19 error-code runbooks following the `AURA-IO-1003` template exactly — same front matter fields, same four required sections, real content in each, no placeholders. Write the four `OPS-*` runbooks. Implement `cargo xtask runbooks --check` and `cargo xtask evidence --check` as specified. Create `PHASE-01-EVIDENCE.md` with all 28 rows, `scripts/collect-evidence.sh`, and `PHASE-01.md` from the template.

Hard constraints: every runbook opens with what the photographer sees, in their language, before any technical explanation; no runbook mentions a Rust type, a crate name or a stack trace above the "For support and engineers" heading; the diagnostics bundle has no unredacted mode; evidence artefacts must record the commit and the check must reject artefacts from a different commit; a row without evidence blocks the phase.

Then run `scripts/collect-evidence.sh` and fill every row you can from this machine. For rows needing the reference machine or a human walkthrough, write the row as `PENDING` with the exact command and who must run it — do not invent a number.

Evidence I expect pasted back: the `runbooks --check` output listing all 19; the `evidence --check` output; the deliberate-failure demonstrations for a missing runbook, an unregistered runbook and a deleted artefact; the full contents listing of a real diagnostics bundle; and the filled `PHASE-01.md` with every measured number in place and every PENDING row explicitly named. Stop after T20.